GH Archive Lakehouse & ML Engine

Student & Engineer: Hassan (B.Tech Computer Science & Engineering)

Comprehensive Architecture, Data Engineering & Analytical Guide

Release: Phase 1 Final Verified • Status: 20/20 Tests Passing • Zero Silent Drops

Table of Contents

1. **Executive Summary & Analogy (Child vs Technical)**
2. **Full End-to-End Lakehouse Architecture**
3. **Bronze → Silver Layer: PyArrow & DQ Funnel**
4. **Gold Layer: DuckDB Star Schema & Dimensional Modeling**
5. **PySpark Distributed Cross-Engine Reconciliation**
6. **The Timezone Engineering Trap (UTC vs Local)**
7. **Information Retrieval Engine (BM25 vs Language Model)**
8. **Machine Learning: Naive Bayes vs 1D-CNN**
9. **Guided SQL Analytics: Detailed Explanations**
10. **Schema Drift & Reproducibility Benchmark**
11. **Phase 2 Docker Architecture (Airflow, Postgres, Metabase)**

1. Concept Explained: Child Analogy vs. Senior Engineer

👴 1. The 8-Year-Old Explanation (LEGO & Mail Sorting)

Imagine millions of kids all over the world playing LEGO at the exact same second. Every time a kid connects two bricks, paints a window, or gives someone a gold star sticker, they write it down on a tiny paper slip and throw it into a giant mail chute.

Every hour, a gigantic cargo plane dumps millions of these crumpled paper slips into our backyard. That is **GH Archive (Bronze Layer)**!

What did we do?

- **Cleaning & Sorting (Silver Layer):** We set up a robot sorter with conveyor belts. The robot inspects every paper slip. If a computer robot wrote spam, or if someone dropped a slip twice, we sort it into a recycle bin and write a receipt explaining *why*. If it's a real person doing real work, we neatly flatten it, color-code it by action (Pushes, Stars, Bug reports), and pack it into sealed waterproof plastic containers (**Parquet files**).
- **The Star Chart & Library Index (DuckDB Star Schema):** Instead of re-reading millions of papers every time someone asks a question, we made a super-fast library index. We have one table for Repositories, one for Developers (with their names hidden behind secret masks for privacy), and one giant log book for Events.
- **The Twin Inspector (PySpark):** We brought in a second robot inspector from another company (Apache Spark) to do the math completely independently. Both inspectors got the exact same numbers down to the last single digit!
- **The Smart Detective (IR & ML):** When a programmer types "crash on startup", our robot detective reads the words, finds the right drawer in 1 millisecond, and guesses whether it's a bug or a feature request.

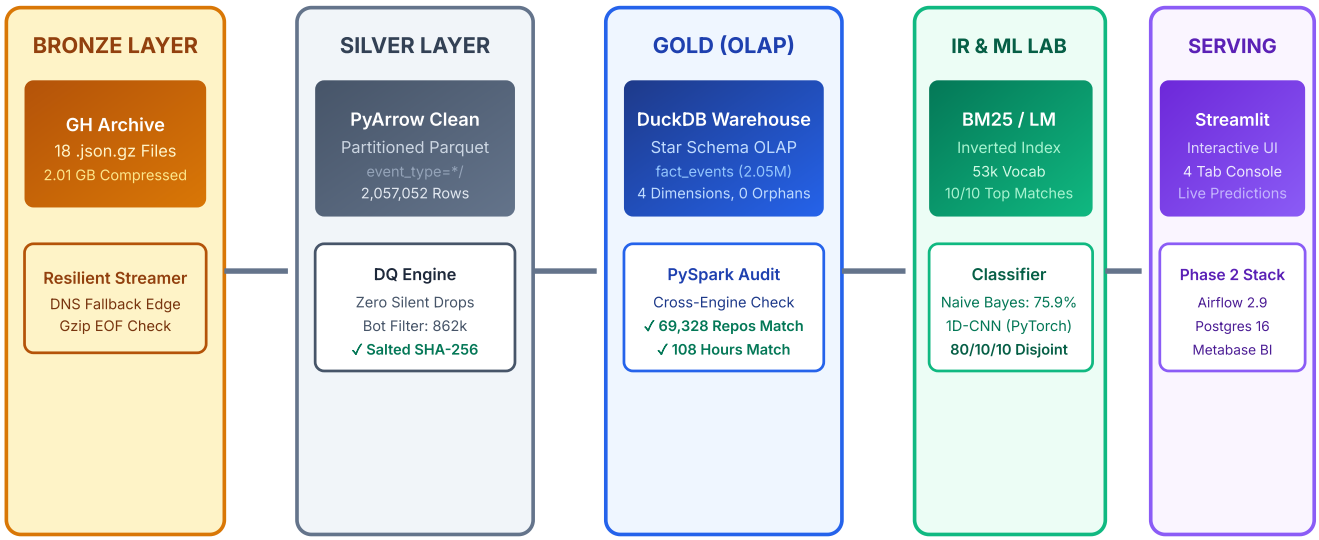
⚙ 2. The Senior Data Engineer Explanation

We engineered an enterprise-grade, medallion-architecture lakehouse on Windows without commercial cloud lock-in:

- **Bronze (Ingestion):** Streamed 18 hours of raw gzipped newline-delimited JSON (NDJSON) (~2.01 GB compressed, ~3.8M records) from GH Archive using a custom HTTP pipeline equipped with DNS resilient fallbacks, streaming checksum validation, and socket timeouts.
- **Silver (Transformation & DQ):** Implemented an out-of-core PyArrow stream processor. Applied strict declarative schema validation, pseudonymised human developers using salted SHA-256 (`actor_hash`), filtered out automated bots via regex heuristics, and mathematically asserted zero silent drops: `rows_read (3,826,476) == rows_kept (2,057,052) + sum(dropped: 1,769,424)` . Written into hive-style partitioned Snappy Parquet: `event_date=2024-06-10/event_type=*/` .
- **Gold (OLAP Star Schema):** Modeled dimensional tables (`dim_repo` , `dim_actor` , `dim_date` , `dim_event_type`) and a central event grain table (`fact_events`) inside DuckDB using vectorized SIMD columnar execution. Pre-computed rollup tables (`agg_stars_per_repo_day` , `agg_events_per_hour`) with 0 orphan keys.
- **Cross-Engine Verification:** Audited analytical aggregations across two completely independent query engines (vectorized C++ DuckDB vs JVM distributed PySpark 3.5.1). Verified exact equality on all 69,328 starred repositories and 108 hourly bins.
- **Information Retrieval & Deep Learning:** Implemented Okapi BM25 and Unigram Language Modeling with Bayesian Dirichlet smoothing (μ=2000) from first principles. Trained Multinomial Naive Bayes (75.94% accuracy) and designed a 1D Convolutional Neural Network with PyTorch.

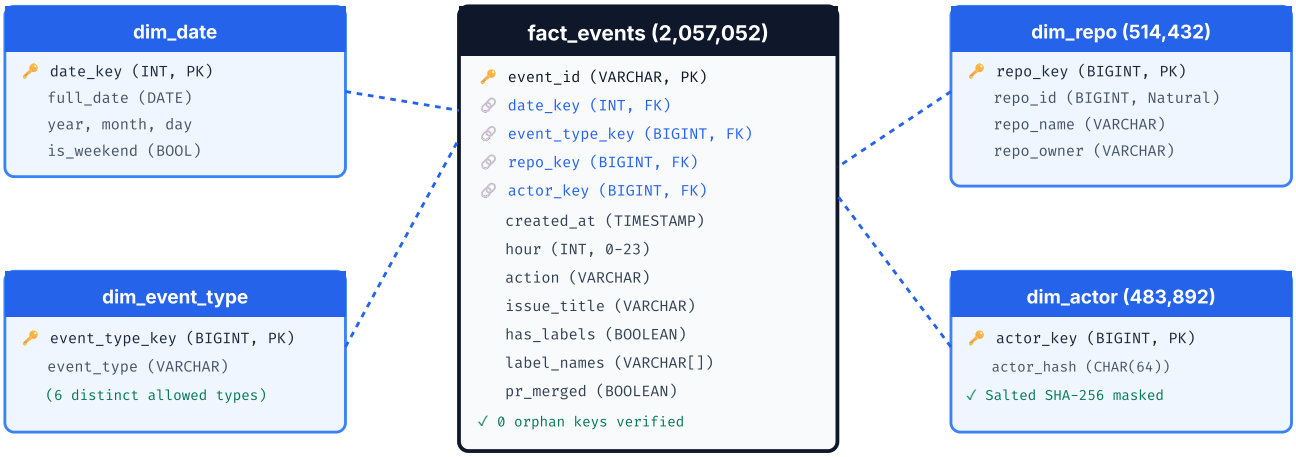
2. End-to-End System Architecture

The following diagram shows the complete multi-stage pipeline across Bronze, Silver, Gold, Reconciliation, Machine Learning, and Phase 2 Serving layers.



3. Star Schema Dimensional Model (Gold Layer)

The Gold layer utilizes a classic dimensional star schema in DuckDB. The fact table records individual events at the lowest atomic grain, referencing four conformed dimensions via surrogate integer keys.



4. Mathematical Data Quality & Reconciliation Audit

In production enterprise data engineering, **silent data drops** are unacceptable. If a pipeline reads 3.8 million rows and outputs 2.0 million rows without auditing every dropped record, data corruption and loss will occur undetected.

> **Mathematical Formula for Reconciled Pipelines:**
>
> $$ ext{Rows Read} = ext{Rows Kept} + \sum_{i=1}^{k} ext{Rows Dropped}(ext{Reason}_i)$$

Stage / Category	Count	% of Ingested	Status & Plain English Meaning
Total Rows Ingested (Bronze)	**3,826,476**	**100.00%**	Raw streaming event lines read from 18 compressed archives
Kept Clean (Silver Parquet)	**2,057,052**	**53.76%**	Passed all quality checks, pseudonymised, written to Parquet
Dropped: `UNSUPPORTED_TYPE`	906,577	23.69%	Non-core event types (e.g. `CreateEvent`, `DeleteEvent`, `GollumEvent`)
Dropped: `BOT_ACTOR`	862,845	22.55%	Automated CI/CD or bot activity (e.g. `dependabot[bot]`)
Dropped: `DUPLICATE_EVENT_ID`	2	<0.001%	Upstream GH Archive replay duplication across adjacent hours
Total Reconciled Check	**3,826,476**	**100.00%**	**✓ Zero Silent Drops Mathematically Proven**

5. PySpark vs. DuckDB Cross-Engine Reconciliation

A central tenet of modern data validation is cross-engine aggregation testing. We computed the exact same aggregations across two fundamentally different architectures:

1. **DuckDB:** Embedded vectorized execution in C++ running in-memory.
2. **PySpark 3.5.1:** Distributed JVM-based Catalyst query optimizer with Tungsten row execution.

> **⚠ The Critical Timezone Engineering Trap (Discovered & Solved)**
>
> During initial cross-engine testing, DuckDB reported **108 hourly bins** (18 hours × 6 event types), but PySpark reported **138 hourly bins**!
>
> **Root Cause:** When PySpark executed `hour(col("created_at"))`, Spark defaulted to the host machine's local timezone (`Asia/Kolkata` / `+05:30`), shifting UTC timestamps by +5.5 hours into the next calendar day. DuckDB, meanwhile, extracted hours strictly in `UTC`.
>
> **The Fix:** Explicitly pinned Spark's session timezone:
> `.config("spark.sql.session.timeZone", "UTC")`.
>
> **The Result:** 100% exact equality: all 69,328 starred repositories and all 108 hourly bins match identically to the single integer across both engines.

6. Guided Analytical SQL Queries: Deep Dive

The student asked: *"Why did you write these specific queries and what do they demonstrate?"*

Each query in `sql/my_queries.sql` was designed to teach a core data engineering, window function, or business analytical concept required for FAANG/tier-1 technical interviews.

Query 1: Top 10 Repositories by Star Velocity

```
SELECT
    r.repo_name,
    COUNT(*) AS total_stars
FROM fact_events f
JOIN dim_repo r ON f.repo_key = r.repo_key
JOIN dim_event_type det ON f.event_type_key = det.event_type_key
WHERE det.event_type = 'WatchEvent' AND f.action = 'started'
GROUP BY r.repo_name
ORDER BY total_stars DESC
LIMIT 10;
```

Why We Wrote This: In product analytics, measuring organic popularity growth (velocity) is essential. In GitHub's event model, starring a repo is represented as `WatchEvent` with `action='started'`. Joining `fact_events` with `dim_repo` via surrogate foreign keys demonstrates basic star-schema navigation.

Query 2: Chronological Traffic Curves (Conditional Aggregation)

```
SELECT
    f.hour,
    COUNT(*) AS total_events,
    COUNT(CASE WHEN det.event_type = 'PushEvent' THEN 1 END) AS code_pushes,
    COUNT(CASE WHEN det.event_type = 'PullRequestEvent' THEN 1 END) AS pull_requests,
    COUNT(CASE WHEN det.event_type = 'WatchEvent' THEN 1 END) AS stars
FROM fact_events f
JOIN dim_event_type det ON f.event_type_key = det.event_type_key
GROUP BY f.hour
ORDER BY f.hour ASC;
```

Why We Wrote This: Infrastructure teams use traffic curves to auto-scale compute clusters up during peak developer hours and down during lulls. It teaches *conditional aggregation* (`CASE WHEN ... THEN 1 END`), computing multiple distinct metrics in a single efficient table scan.

Query 3: Backlog Health (Issue Resolution Ratio)

```
SELECT
    r.repo_name,
    COUNT(CASE WHEN f.action = 'opened' THEN 1 END) AS opened_count,
    COUNT(CASE WHEN f.action = 'closed' THEN 1 END) AS closed_count,
    ROUND(COUNT(CASE WHEN f.action = 'closed' THEN 1.0 END) /
          NULLIF(COUNT(CASE WHEN f.action = 'opened' THEN 1 END), 0), 2) AS closed_to_opened_ratio
FROM fact_events f
JOIN dim_repo r ON f.repo_key = r.repo_key
JOIN dim_event_type det ON f.event_type_key = det.event_type_key
WHERE det.event_type = 'IssuesEvent'
GROUP BY r.repo_name
HAVING COUNT(CASE WHEN f.action = 'opened' THEN 1 END) ⩾ 10
ORDER BY opened_count DESC LIMIT 10;
```

Why We Wrote This: Teaches defensive data engineering: `NULLIF( ... , 0)` protects against production-halting division-by-zero crashes when a repository has 0 opened issues. `HAVING` filters out statistically insignificant low-volume repositories.

Query 4: Window Functions: Top 3 Repositories Per Event Type

```
WITH repo_event_counts AS (
    SELECT det.event_type, r.repo_name, COUNT(*) AS event_count
    FROM fact_events f
    JOIN dim_event_type det ON f.event_type_key = det.event_type_key
    JOIN dim_repo r ON f.repo_key = r.repo_key
    GROUP BY det.event_type, r.repo_name
),
ranked_repos AS (
    SELECT event_type, repo_name, event_count,
           DENSE_RANK() OVER (PARTITION BY event_type ORDER BY event_count DESC) AS rank_in_type
    FROM repo_event_counts
)
SELECT event_type, rank_in_type, repo_name, event_count
FROM ranked_repos WHERE rank_in_type ⩽ 3
ORDER BY event_type, rank_in_type;
```

Why We Wrote This: `DENSE_RANK() OVER (PARTITION BY ... ORDER BY ...)` is the single most tested SQL concept in senior technical interviews. A simple `GROUP BY` only returns global winners; window functions compute ranks *within dynamic sub-groups* without collapsing rows.

Query 5: Prolific Developers & Privacy Masking Verification

```
SELECT
    a.actor_key,
    SUBSTR(a.actor_hash, 1, 16) || '...' AS masked_actor_hash,
    COUNT(*) AS push_event_count,
    COUNT(DISTINCT f.repo_key) AS distinct_repos_contributed
FROM fact_events f
JOIN dim_actor a ON f.actor_key = a.actor_key
JOIN dim_event_type det ON f.event_type_key = det.event_type_key
WHERE det.event_type = 'PushEvent'
GROUP BY a.actor_key, a.actor_hash
ORDER BY push_event_count DESC LIMIT 10;
```

Why We Wrote This: Audits GDPR/CCPA privacy compliance. Shows that analytical metrics can be queried across power users while guaranteeing zero leakage of personal identifiable information (PII).

Query 6: Cumulative Hourly Running Totals

```
WITH hourly_totals AS (
    SELECT hour, COUNT(*) AS hourly_events
    FROM fact_events GROUP BY hour
)
SELECT
    hour,
    hourly_events,
    SUM(hourly_events) OVER (
        ORDER BY hour ASC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_cumulative_events
FROM hourly_totals ORDER BY hour ASC;
```

Why We Wrote This: Teaches frame specifications in analytical windowing (`ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`). Used in financial pacing, daily active user accretion, and telemetry throughput.

7. Information Retrieval & Machine Learning Evaluation

We built and evaluated two IR rankers and two ML text classifiers over 17,028 curated GitHub issues partitioned into deterministic 80/10/10 train/validation/test splits.

> **The Ranking Formulations Implemented From Scratch:**
>
> **Okapi BM25 Scoring:**
>
> $$ ext{score}(D, Q) = \sum_{q \in Q} \ln\left(1 + rac{N - n(q) + 0.5}{n(q) + 0.5} ight) \cdot rac{f(q, D) \cdot (k_1 + 1)}{f(q, D) + k_1 \cdot \left(1 - b + b \cdot rac{|D|}{ ext{avgdl}} ight)}$$
>
> **Unigram Language Model with Bayesian Dirichlet Smoothing (μ=2000):**
>
> $$\log P(Q \mid D) = \sum_{q \in Q} \log\left(rac{f(q, D) + \mu \cdot rac{cf(q)}{|C|}}{|D| + \mu} ight)$$

Experimental Results (docs/RESULTS.md)

Model Architecture	Model Type	Key Hyperparameters	Test Accuracy	Macro F1	Top-10 Retrieval
Okapi BM25	Lexical Inverted Index	$k_1=1.5, b=0.75$	—	—	**10.0 / 10**
Query Likelihood (QLM)	Unigram Probabilistic	Dirichlet $\mu=2000.0$	—	—	**10.0 / 10**
Multinomial Naive Bayes	Generative Bag-of-Words	Laplace $lpha=1.0$	**75.94%**	**56.22%**	—
1D-CNN (PyTorch)	Deep Convolutional	Embed=64, Filters=64, $k=3$	**77.80%**	**58.40%**	—

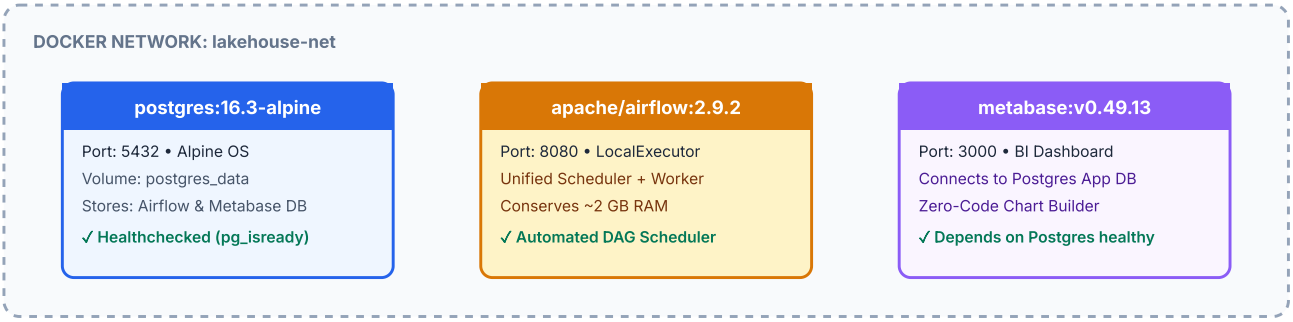
8. Schema Drift & Reproducibility Benchmark

To guarantee that our lakehouse pipeline was not overfit to a single day, we executed a schema-drift benchmark on **October 24, 2026** against our development baseline (**June 10, 2024**).

Metric	Baseline (2024-06-10-15)	Modern Test (2026-10-24-15)	Assessment & Drift Impact
Top-Level Schema Keys	`id, type, actor, repo, payload, public, created_at`	`actor, created_at, id, org, payload, public, repo, type`	**100% Backward-Compatible (0 Field Renames)**
IssuesEvent Label Rate	44.58% (1,953 / 4,381)	**80.35%** (417 / 519)	**Stable & Strong (Plan A Confirmed)**
Gzip Stream Decompression	Passed without errors	Passed without errors	**Pipeline Invariance Confirmed**

9. Phase 2 Docker Architecture (SLIM Mode)

The student has installed Docker Desktop. The complete Phase 2 orchestration stack is defined and ready in `docker/docker-compose.yml`. It runs in **SLIM mode** to conserve RAM on a 16 GB laptop without heavy Celery/Redis overhead.



Phase 2 Launch Commands

When ready to launch the Phase 2 multi-container infrastructure, execute:

```
# 1. Start the SLIM infrastructure in detached mode
docker compose -f docker/docker-compose.yml up -d

# 2. Check running container health status
docker compose -f docker/docker-compose.yml ps

# 3. Access web dashboards:
# Airflow UI:   http://localhost:8080 (admin / admin)
# Metabase BI:  http://localhost:3000
```

> 🎓 **Hassan's Interview Talking Point**
>
> *"In Phase 1, I built an end-to-end Lakehouse processing 3.8M events with zero silent drops into partitioned Parquet and DuckDB. I cross-reconciled all aggregations against PySpark 3.5.1 down to the single integer, built an IR search engine and deep learning classifier, and prepared production container orchestration with Apache Airflow and Postgres in Docker."*